AI USAGE

For the following error:

==========

TypeError Traceback (most recent call last)

Cell In[5], line 33

31 current_data = pl.from_pandas(current_data)

32 for row in current_data.iter_rows():

---> 33 symbol = row['symbol']

34 price = row['close']

35 signal = row['signal']

TypeError: tuple indices must be integers or slices, not str

==========

Prompt:

“

TypeError Traceback (most recent call last)

Cell In[5], line 33

31 current_data = pl.from_pandas(current_data)

32 for row in current_data.iter_rows():

---> 33 symbol = row['symbol']

34 price = row['close']

35 signal = row['signal']

TypeError: tuple indices must be integers or slices, not str

any reason why this error occured? Please tell from where is this error coming from?

“

The error is occurring because you're trying to access `row` as if it were a dictionary (using string keys), but by default `iter_rows()` in Polars returns a tuple of values, not a dictionary.

Fix the error:

Change this:

```
for row in current_data.iter_rows():
    symbol = row['symbol']  # This causes error - can'
```

To this:

```
for row in current_data.iter_rows(named=True):  # Add
    symbol = row['symbol']  # Now works correctly
```

Explanation:

- In Polars, `iter_rows()` returns tuples by default (accessed by position/index)
- `iter_rows(named=True)` returns dictionary-like objects (accessed by column name)

Your implementation correctly handles:

- Pyramiding with 50 shares per signal
- Max position size of 1,000 shares
- Max daily loss limit of 2%
- Exiting all positions at reverse signals

This change will allow your backtest to run as intended while maintaining all your risk control logic.

‘’’

Went with Co pilot’s suggestion as named = True let’s me access the rows as dicts instead of tuples.

Prompt:

“create pnl curve, win rate, sharpe ratio, average returns, max returns, min returns, max drawdown for both stressed and unstressed backtests.”

Output:

Performance Analysis for Original and Stressed Backtests

Let's create a comprehensive performance analysis for both the original and stressed backtest results:

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.dates import DateFormatter
import pandas as pd

def calculate_performance_metrics(trades_df, name="Backtest"):
    """Calculate key performance metrics from trade data"""
    metrics = {}

    # If no trades or no PnL data, return empty metrics
    if len(trades_df) == 0 or 'pnl' not in trades_df.columns:
        print(f"No trade data available for {name}")
        return metrics

    # Convert to pandas for easier datetime handling
    df = trades_df.to_pandas()
    df['timestamp'] = pd.to_datetime(df['timestamp'])

    # Filter to only completed trades (with PnL)
    completed_trades = df[df['pnl'].notnull()]

    # Basic metrics
    metrics['total_trades'] = len(completed_trades)
    metrics['profitable_trades'] = len(completed_trades[completed_trades['pnl'] > 0])
```

```
    metrics['win_rate'] = metrics['profitable_trades'] / metrics['total_trades'] if metrics['total_trades'] > 0 else 0

    # Return metrics
    metrics['total_pnl'] = completed_trades['pnl'].sum()
    metrics['avg_return'] = completed_trades['pnl'].mean()
    metrics['max_return'] = completed_trades['pnl'].max()
    metrics['min_return'] = completed_trades['pnl'].min()

    # Sharpe ratio (assuming daily, annualized)
    daily_returns = completed_trades.groupby(completed_trades['timestamp'].dt.date)['pnl'].sum()
    metrics['sharpe_ratio'] = (daily_returns.mean() / daily_returns.std()) * np.sqrt(252) if len(daily_returns) > 1 else 0

    # Max drawdown calculation
    if 'cumulative_pnl' in completed_trades.columns:
        cumulative = completed_trades.sort_values('timestamp')
        running_max = cumulative['cumulative_pnl'].cummax()
        drawdown = (cumulative['cumulative_pnl'] - running_max) / running_max if running_max.max() > 0 else 0
        metrics['max_drawdown'] = drawdown.min() if isinstance(drawdown, pd.Series) else 0
    else:
        metrics['max_drawdown'] = 0

    return metrics

# Calculate metrics for both datasets
original_metrics = calculate_performance_metrics(trades_df, "Original Backtest")
stressed_metrics = calculate_performance_metrics(trades_df, "Stressed Backtest (-5% shock)")

# Display metrics in a table
metrics_comparison = pd.DataFrame({
    'Metric': ['Total Trades', 'Win Rate (%)', 'Sharpe Ratio', 'Average Return ($)',
```

```
            'Max Return ($)', 'Min Return ($)', 'Max Drawdown (%)', 'Total P&L ($)'],
    'Original': [original_metrics.get('total_trades', 0),
                 f"{original_metrics.get('win_rate', 0)*100:.2f}%",
                 f"{original_metrics.get('sharpe_ratio', 0):.2f}",
                 f"${original_metrics.get('avg_return', 0):.2f}",
                 f"${original_metrics.get('max_return', 0):.2f}",
                 f"${original_metrics.get('min_return', 0):.2f}",
                 f"{original_metrics.get('max_drawdown', 0)*100:.2f}%",
                 f"${original_metrics.get('total_pnl', 0):.2f}"],
    'Stressed (-5%)': [stressed_metrics.get('total_trades', 0),
                       f"{stressed_metrics.get('win_rate', 0)*100:.2f}%",
                       f"{stressed_metrics.get('sharpe_ratio', 0):.2f}",
                       f"${stressed_metrics.get('avg_return', 0):.2f}",
                       f"${stressed_metrics.get('max_return', 0):.2f}",
                       f"${stressed_metrics.get('min_return', 0):.2f}",
                       f"{stressed_metrics.get('max_drawdown', 0)*100:.2f}%",
                       f"${stressed_metrics.get('total_pnl', 0):.2f}"]
})

print(metrics_comparison.to_string(index=False))

# Create P&L curve visualizations
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))

# Plot PnL curves if data exists
if len(trades_df) > 0 and 'cumulative_pnl' in trades_df.columns:
    df1 = trades_df.to_pandas()
    df1['timestamp'] = pd.to_datetime(df1['timestamp'])
    df1 = df1.sort_values('timestamp')

    ax1.plot(df1['timestamp'], df1['cumulative_pnl'], 'b-', linewidth=2)
```

```
    ax1.set_title('Original Backtest: Cumulative P&L', fontsize=14)
    ax1.set_ylabel('P&L ($)', fontsize=12)
    ax1.grid(True)
    ax1.xaxis.set_major_formatter(DateFormatter('%H:%M'))

# Stressed backtest PnL
if len(trades_df) > 0 and 'cumulative_pnl' in trades_df.columns:
    df2 = trades_df.to_pandas()
    df2['timestamp'] = pd.to_datetime(df2['timestamp'])
    df2 = df2.sort_values('timestamp')

    ax2.plot(df2['timestamp'], df2['cumulative_pnl'], 'r-', linewidth=2)
    ax2.set_title('Stressed Backtest (-5% Shock): Cumulative P&L', fontsize=14)
    ax2.set_xlabel('Time', fontsize=12)
    ax2.set_ylabel('P&L ($)', fontsize=12)
    ax2.grid(True)
    ax2.xaxis.set_major_formatter(DateFormatter('%H:%M'))

plt.tight_layout()
plt.show()

# Plot a comparison of drawdowns
if len(trades_df) > 0 and 'cumulative_pnl' in trades_df.columns:
    # Calculate drawdowns for both datasets
    def calculate_drawdowns(df):
        df = df.sort_values('timestamp')
        df['peak'] = df['cumulative_pnl'].cummax()
        df['drawdown'] = (df['cumulative_pnl'] - df['peak']) / df['peak'] * 100
        return df

    orig_dd = calculate_drawdowns(trades_df.to_pandas())
```

```
stress_dd = calculate_drawdowns(trades_df.to_pandas())


plt.figure(figsize=(14, 6))
plt.plot(orig_dd['timestamp'], orig_dd['drawdown'], 'b-', label='Original')
plt.plot(stress_dd['timestamp'], stress_dd['drawdown'], 'r-', label='Stressed (-5%)')
plt.title('Drawdown Comparison', fontsize=14)
plt.xlabel('Time', fontsize=12)
plt.ylabel('Drawdown (%)', fontsize=12)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

My Interpretation:
Here everything is okay except for Sharpe ratio. As our backtest result is on 1 minute interval and that too just for a dataset of 1 day and 3 stocks. So Co-pilot's sharpe ratio formula is not fit here. So we need to adjust for our minute level pnl curve.

"

if len(completed_trades) > 1:

minute_returns = completed_trades.groupby(completed_trades['timestamp'].dt.floor('min'))['pnl'].sum()

if minute_returns.std() != 0:

metrics['intraday_sharpe'] = (minute_returns.mean() / minute_returns.std()) * np.sqrt(len(minute_returns)) -> minute returns = 376 in here.

"

Scaling to Production with AWS

1. Data Storage & Management

- **Tick Data Ingestion:**
    - Store raw tick data in a high-performance OLAP database such as **ClickHouse**.
    - Organize tables by exchange, instrument type, and date for efficient querying and partitioning.
- **Resampled Data Storage:**
    - At EOD, resample tick data to 1-minute (or other) bars.
    - Store resampled OHLCV data as Parquet files in **AWS S3**, partitioned by exchange/instrument_type/year/month/date.parquet for scalable, cost-effective storage and fast retrieval.
- **Data Catalog & Query:**
    - Use **AWS Glue** crawlers to automatically catalog new Parquet data in S3.
    - Query historical and resampled data using **AWS Athena** for ad-hoc analysis, reporting, and backtesting.

---

2. Data Access Layer & APIs

- **Service Layer:**
    - Build a **FastAPI** service layer to expose endpoints for data access, backtesting, and live trading.
    - The service layer routes queries to Athena (for historical/resampled data) or ClickHouse (for real-time tick data), depending on the requested interval and time range.
- **API Endpoints:**
    - Provide endpoints for:
        - Data retrieval (tick, minute, EOD)
        - Backtest execution
        - Live trading signal generation
        - Position and PnL monitoring

---

3. Trading System Architecture

- **Pre-Trade Infrastructure:**
    - **Rate Limiters:** Enforce API and trading limits to comply with exchange and broker requirements.
    - **Data Feeders:** Distribute real-time price data to multiple system components for redundancy and low-latency processing.
- **Core Trading System:**
    - **OMS (Order Management System):** Handles order placement, modification, and cancellation.
    - **RMS (Risk Management System):** Enforces position limits, max daily loss, and other risk controls in real time.
    - **Strategy Management:**
        - Support for multiple, config-driven strategies.
        - Centralized configuration for parameters, rate limits, and standardized order variables.
    - **Caching:**
        - Use **Redis** for caching frequently accessed data (e.g., latest prices, positions, signals) to reduce latency.
    - **Persistence:**
        - Save EOD positions, trades, and signals to a database for audit and recovery.
- **Event-Driven Processing:**
    - The system operates on time slots (e.g., every minute or tick).
    - Each slot triggers:
        - Data ingestion and signal computation
        - Position and PnL updates
        - Order and risk checks

---

4. Scalability & Reliability

- **Horizontal Scaling:**
    - Use AWS EC2 Auto Scaling Groups or AWS ECS for stateless service scaling.
    - Partition workloads by symbol, exchange, or strategy for parallel processing.
- **Fault Tolerance:**
    - Use S3 versioning and multi-AZ ClickHouse clusters for data durability.
    - Implement retry and failover logic in the service and trading layers.
- **Monitoring & Alerting:**
    - Integrate with **AWS CloudWatch** for system health, latency, and error monitoring.
    - Set up alerts for risk breaches, system failures, and abnormal trading activity.

---

5. Security & Compliance

- **IAM Roles & Policies:**
    - Restrict access to S3, Athena, ClickHouse, and trading APIs using AWS IAM.
- **Audit Logging:**
    - Log all API calls, order actions, and risk events for compliance and traceability.

---

6. Extensibility

- **Config-Driven Strategies:**
    - Manage all strategy parameters, risk limits, and operational settings via YAML/JSON configs.
    - Support hot-reloading of configs for rapid iteration and deployment.
- **Plug-and-Play Components:**

- Design the system to easily add new strategies, data sources, or risk checks without major refactoring.

Summary:
This AWS-centric, modular, event-driven architecture ensures your trading system is robust, scalable, and ready for both research and production. It leverages cloud-native storage, compute, and analytics, while maintaining low-latency and high reliability for live trading.

This is Co pilot generated on this prompt:

Now let's think of scaling this up.

- Next steps (how you'd scale to production with Databricks/AWS)

these are the components of a trading system

1. Database for storing the tick data

tick data in the OLAP clickhouse DB in an organized manner, detail on this.

resampled data EOD in AWS S3 partitioned by exchange/ instrument type/ year/ month/ date.parquet

2. Create table in AWS Glue using crawler and query data using AWS Athena

get the data from fastapi endpoints, below that there will be a service layer that hits the athena or Clickhouse db depending upon the interval, from date, to date.

endpoints can be used for backtesting and live trading system

3. now comes the trading system

before OMS and RMS comes Rate limiters, datafeeder that replicates the price data on multiple systems.

the trading system consists for OMS, RMS, manangement of multiple strategies, caching the data required on regular basis on redis, saving EOD positions in DB and signals for next day to entry and exit.

config driven system for managing the params for multiple strategies, for rate limits, defining standardized variables for order placing, and other functions.

the system will be event driven where system will work on time slots for trading, processing the data for signals, processing positions, updating the PNL. Now I need you to document this process in a neat manner.

I do not use Databricks, But use AWS for large scale computing using polars and duckDB leveraging their parallel computing capabilities.